GHC.Conc: https://hackage.haskell.org/package/base/docs/GHC-Conc.html
Control.Concurrent.STM: https://hackage.haskell.org/package/stm
STM

```
-- Control.Concurrent.STM
- Base in GHC.Conc
data STM a
atomically :: STM a -> IO a
retry :: STM a
orElse :: STM a -> STM a -> STM a
check :: Bool -> STM ()
throwSTM :: Exception e => e -> STM a
catchSTM :: Exception e => STM a -> (e -> STM a) -> STM
a
```

Control.Concurrent.STM

```
-- Control.Concurrent.STM.TSem
data TSem
newTSem :: Integer -> STM TSem
waitTSem :: TSem -> STM ()                              --
acquire lock
signalTSem :: TSem -> STM ()                            --
release lock
signalTSemN :: Natural -> TSem -> STM ()


-- Control.Concurrent.STM.TVar
data TVar a
newTVar :: a -> STM (TVar a)
newTVarIO :: a -> IO (Tvar a)
readTVar :: TVar a -> STM a
  -- read shared memory
readTVarIO :: TVar a -> IO a
writeTVar :: TVar a -> a -> STM ()
   -- write shared memory
modifyTVar :: TVar a -> (a -> a) -> STM ()
stateTVar :: TVar s -> (a -> (a, s)) -> STM a
swapTVar :: TVar a -> a -> STM a
registerDelay :: Int -> IO (TVar Bool)
mkWeakTVar :: TVar a -> IO () -> IO (Weak (TVar a))


-- Control.Concurrent.STM.TMVar
data TMVar a
newTMVar :: a -> STM (TMVar a)
newEmptyTMVar :: STM (TMVar a)
newTMVarIO :: a -> IO (TMVar a)
newEmptyTMVarIO :: IO (TMVar a)
takeTMVar :: TMVar a -> STM a                     -- take
value and block
```

```
putTMVar :: TMVar a -> a -> STM ()                -- put
new value and free
readTMVar :: TMVar a -> STM a                     -- take
+ put

-- non-blocking
writeTMVar :: TMVar a -> a -> STM ()              -- non-
blocking force put
swapTMVar :: TMVar a -> a -> STM a
tryTakeTMVar:: TMVar a -> STM (Maybe a)
tryPutTMVar :: TMVar a -> a -> STM Bool
tryReadTMVar :: TMVar a -> STM (Maybe a)
isEmptyTMVar :: TMVar a -> STM Bool
```

Software Transaction Memory (STM)
Types of memory
- **TSem**: semaphore i.e. mutual lock
  actions: **acquire** lock + **release** lock

- **TMVar**: like value with lock
  i.e. if a process reads the value, others are **blocked** from using the value until the process puts it back.
  actions: **take** value and block (empty) + **put** value and free (full).

- **TVar**: shared memory.
 actions: **read** value + **write** value
 Unlike TMVar, no lock

Example: A simple lock used by at most 1 process at a time

```
import Control.Concurrent.STM.TSem
import Control.Concurrent.STM
import Control.Exception (bracket_)
import Control.Concurrent (forkIO, threadDelay)

-- Create a lock which can be used by at most 1 process
createLock :: IO TSem
createLock = atomically $ newTSem 1

runAction :: TSem -> IO a -> IO a
runAction sem = bracket_
  (atomically $ waitTSem sem)                      --
acquire lock
  (atomically $ signalTSem sem)                    --
free lock

wait :: Int
wait = 2 * 1000 * 1000

startTest :: IO TSem
startTest = createLock >>= someAction "Start"
```

```
someAction :: String -> TSem -> IO TSem
someAction msg sem = do
  _ <- forkIO $ runAction sem $ threadDelay wait >>
putStrLn msg
  return sem
```

e.g.

```
ghci> startTest >>= someAction "a" >>= someAction "b"
ghci>
Start
.. 2 seconds ..
a
.. 2 seconds ..
b
```

Example: lock used by n processes at once

```
import Control.Concurrent
import Control.Concurrent.STM
import Control.Concurrent.STM.TSem
import Control.Monad

waitTime :: Int
waitTime = 2 * 1000 * 1000

afterDelay :: Int -> IO () -> IO ()
afterDelay n action = threadDelay n >> action

-- run action when lock is free
runAction :: TSem -> IO () -> IO ()
runAction sem action =
  void $
    forkIO $
      atomically (waitTSem sem) >> action           --
wait for lock

-- signal free lock
signalFree :: TSem -> IO ()
signalFree sem = atomically $ signalTSem sem        --
free 1 lock

test :: IO ()
test = do
  -- initially at most 3 processes at once
  sem <- atomically $ newTSem 3

  -- 3 in  parallel
  runAction sem $ afterDelay waitTime $ putStrLn "a"
  runAction sem $ afterDelay waitTime $ putStrLn "b"
  runAction sem $ afterDelay waitTime $ putStrLn "c" >>
signalFree sem
```

```
  -- sequential
  runAction sem $ afterDelay waitTime $ putStrLn "d" >>
signalFree sem       -- free lock for next process
  runAction sem $ afterDelay waitTime $ putStrLn "e" >>
signalFree sem       -- free lock for next process
  runAction sem $ afterDelay waitTime $ putStrLn "f"
    >> signalFree sem >> signalFree sem >> signalFree
sem                    -- free 3 locks for parallel

  -- 3 in parallel
  runAction sem $ afterDelay waitTime $ putStrLn "g"
  runAction sem $ afterDelay waitTime $ putStrLn "h"
  runAction sem $ afterDelay waitTime $ putStrLn "i"

  -- never run
  runAction sem $ afterDelay waitTime $ putStrLn "z"
```

e.g.

```
ghci> test
... 2 seconds ...
cba
... 2 seconds ...
d
... 2 seconds ...
e
... 2 seconds ...
f
... 2 seconds ...
ghi
```